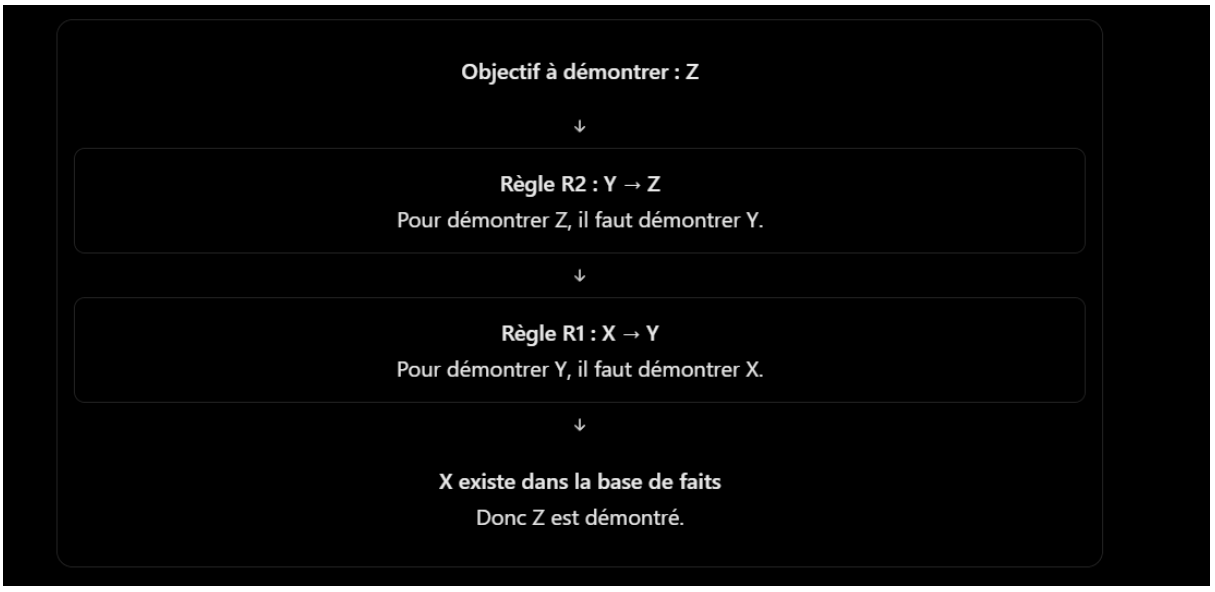
chaînage arrière

Nous venons de voir que le chaînage avant part des faits connus pour découvrir toutes les conclusions possibles.

Le chaînage arrière fait l'inverse : il part d'une conclusion qu'on veut démontrer et cherche les faits nécessaires pour la justifier.

2. Définition

Le chaînage arrière est une méthode d'inférence qui part d'un objectif et remonte les règles logiques jusqu'aux faits connus pour vérifier si cet objectif peut être démontré.



Différence essentielle :

- Chaînage avant : Qu'est-ce que je peux déduire de mes faits ?
- Chaînage arrière : Est-ce que je peux démontrer cet objectif avec mes faits ?

3. Exercice — Diagnostic informatique

Nous conservons le même exemple pour faciliter la comparaison.

Base de faits :

- `ORNDP` : l'ordinateur ne démarre pas.

- `TBV` : la batterie est vide (confirmée par un test).

Base de règles :

- R1 : `TBV → MDN`
- R2 : `MDN → BRCH`

Objectif : démontrer `BRCH` (il faut brancher le chargeur).

Questions

1. Quelle règle permet de démontrer `BRCH` ?
2. Quel sous-objectif faut-il démontrer pour appliquer cette règle ?
3. Peut-on finalement démontrer `BRCH` à partir des faits initiaux ?
4. Programmer en Python un chaînage arrière qui vérifie automatiquement un objectif.

4. Résolution de l'exercice

Objectif : démontrer BRCH.

Nous raisonnons à partir de l'objectif, en remontant les règles.

1. **BRCH** est la conclusion de R2 : `MDN → BRCH`. Il faut donc démontrer MDN.
2. **MDN** est la conclusion de R1 : `TBV → MDN`. Il faut donc démontrer TBV.
3. **TBV** appartient à la base de faits. Il est donc connu.

Nous pouvons alors démontrer MDN, puis BRCH.

Conclusion : BRCH est démontré.

5. Implémentation Python

Voir le notebook de l'exercice de chaînage arrière.

Le programme repose sur une fonction récursive :

```
def chainage_arriere(objectif):

    # Vérifier si l'objectif est un fait connu
    if objectif in faits:
        return True

    # Chercher une règle qui démontre l'objectif
    for regle in regles:
```

```
        condition, conclusion = regle.args

        if conclusion == objectif:
            # Vérifier le sous-objectif
            if chainage_arriere(condition):
                return True

    return False
```

Comprendre la récursivité

La récursivité signifie qu'une fonction peut **s'appeler elle-même**.

Dans notre exercice :

`chainage_arriere(BRCH)` → `chainage_arriere(MDN)` → `chainage_arriere(TBV)`

Puisque TBV est un fait connu, la fonction retourne `True`. Ce résultat remonte jusqu'à BRCH.

Résultat : l'objectif est démontré.

6. Limites

- Si aucune règle ni aucun fait ne permet de démontrer l'objectif, le programme retourne `False`.
- `False` signifie que l'objectif n'est pas démontrable avec les connaissances disponibles, pas nécessairement qu'il est faux.
- Notre implémentation traite des règles simples. Des règles circulaires nécessitent une protection contre les appels récursifs infinis.

7. À retenir

Chaînage avant : Faits → Règles → Conclusions.

Chaînage arrière : Objectif → Règles → Sous-objectifs → Faits connus.

Le chaînage arrière est donc un **raisonnement dirigé par les objectifs**, utilisé notamment dans les systèmes experts pour vérifier une hypothèse.

Idée essentielle : pour démontrer une conclusion, on cherche les conditions nécessaires, puis on vérifie si elles peuvent être démontrées à partir des faits connus.